

PLANTEL LISTAS

Manual Tecnico Oficial - V1

Arquitetura, regras de negocio, seguranca, permissoes e metodologia de implementacao

Versao 1.0 | 26 de agosto de 2026

Status: V1 funcionalmente congelada

1. Finalidade do documento

Este manual e a referencia tecnica oficial da V1 do Plantel Listas. Ele deve permanecer no repositório e ser lido pelo Codex antes de qualquer implementacao. O objetivo e impedir perda de contexto, mudancas acidentais de regra de negocio e decisoes arquiteturais inconsistentes quando houver troca de conta, chat ou sessao.

A documentacao e parte do sistema. O codigo deve obedecer aos documentos; o chat do Codex nao e a fonte de verdade.

2. Visao do produto

O Plantel Listas transforma o acervo hoje armazenado no Google Drive em uma experiencia propria, profissional, pesquisavel e navegavel para alunos, professores e administradores.

- Google Drive continua armazenando os PDFs e videos.
- MySQL armazena usuarios, permissoes, classificacoes, metadados, analytics e auditoria.
- O frontend entrega a experiencia de navegacao.
- O backend concentra as regras de negocio, autorizacao e integracoes.

Principio: Drive armazena. Banco descreve. Plantel Listas entrega a experiencia.

3. Escopo congelado da V1

- Cadastro de aluno, login e logout.
- Login de professor e admin.
- Recuperacao e redefinicao de senha por email.
- Roles: aluno, professor e admin.
- Categorias e subcategorias.
- Acervo de PDFs e videos.
- Busca e filtros por metadados relevantes.
- Visualizacao de PDF, reproducao de video e download.
- Upload por professor/admin.
- Edicao de metadados, movimentacao e substituicao de arquivo.
- Permissao de professor limitada a areas liberadas pelo admin.
- Lixeira e restauracao/exclusao administrativa.
- Integracao com Google Drive.
- Importacao/indexacao inicial do acervo existente.
- Dashboard administrativo de analytics.
- Auditoria de acoes sensiveis.
- Interface responsiva.
- Testes funcionais, de integracao, banco, autorizacao e ciberseguranca.
- Deploy em producao na Hostinger.

4. Fora da V1 e roadmap

4.1 V2 - Banco de Questoes

Modulo futuro independente do acervo documental da V1.

4.2 V2.x - Recursos avancados

- listas personalizadas;
- salvar questoes;
- historico;
- questoes ja resolvidas;

- geracao de simulados;
- estatisticas de desempenho.

4.3 V3 - Modulo comercial

- planos/premium;
- pagamentos;
- PIX/cartao;
- beneficios e recursos por plano;
- gestao de assinatura;
- webhooks;
- controle financeiro.

O Codex nao deve implementar nem antecipar complexidade desses modulos na V1. A arquitetura deve apenas evitar decisoes que tornem a evolucao futura desnecessariamente dificil.

5. Arquitetura oficial

Camada	Tecnologia / responsabilidade
Frontend	React + Vite + JavaScript
Backend	Node.js + Express + CommonJS
Banco	MySQL / InnoDB / utf8mb4
Arquivos	Google Drive via Drive API
Local	Laragon + MySQL + MySQL Workbench
Producao	Hostinger Business Web Hosting

Preferencia de implantacao: frontend e API na mesma origem sempre que viavel. O Express pode servir o build do React, reduzindo CORS e simplificando cookies e sessoes.

5.1 Estrutura de responsabilidades

- Routes: endpoint + middlewares + controller.
- Controller: HTTP; nao contem regra de negocio.
- Service: regra de negocio e coordenacao.
- Repository/Model: acesso MySQL por SQL parametrizado.
- Middleware: autenticao, autorizacao, CSRF, rate limit e validacoes transversais.
- Provider: integracoes externas, especialmente Google Drive.
- Validator: formato dos dados de entrada.

5.2 Ambiente de producao

O plano informado e Hostinger Business Web Hosting. A Hostinger atualmente suporta aplicacoes Node.js/Express nesse plano e oferece MySQL gerenciado. O deploy deve usar variaveis de ambiente, HTTPS e fluxo versionado por Git.

A V1 nao requer VPS, Docker ou PostgreSQL.

6. Google Drive

Pasta raiz confirmada para o acervo:

10Kokm2f3Ipe0FuzIJvJDc4HpHZpBo0IX

Raiz observada em 26/08/2026:

- LISTAS
- PROVAS ANTIGAS

A estrutura atual deve ser importada/indexada, mas nunca hard-coded no frontend. O sistema deve suportar evolucao de categorias sem depender de nomes fixos do Drive.

6.1 Integracao

- Conta Google do acervo e Gmail comum.
- A aplicacao nao usa nem armazena a senha da conta Google.
- Integracao via Google Drive API e OAuth 2.0.
- Somente a conta designada para o acervo concede acesso Google.
- Credenciais OAuth e refresh token ficam exclusivamente no backend/ambiente seguro.
- Provider Google Drive isolado.

Principio de menor privilegio: a fase de integracao deve validar primeiro o menor escopo OAuth que atende ao acervo legado e as operacoes de escrita. Caso seja necessario um escopo amplo do Drive, o Codex deve documentar a necessidade e solicitar aprovacao antes de adota-lo.

6.2 Sincronizacao

- Executar importacao/indexacao inicial do acervo existente.
- Usar `drive\_file\_id` como identificador externo unico.
- Processo de indexacao deve ser idempotente.
- Registrar execucoes de sincronizacao.
- Disponibilizar reindexacao administrativa quando necessaria.
- Depois do go-live, operacoes normais de acervo devem ocorrer preferencialmente pelo Plantel Listas.

7. Usuarios, autenticao e permissoes

Existem somente tres roles na V1: aluno, professor e admin.

Acao	Aluno	Professor	Admin
Cadastrar-se	Sim	Nao como professor	Nao como admin
Acessar/baixar	Sim	Sim	Sim
Upload	Nao	Area autorizada	Sim
Editar/mover/substituir	Nao	Area autorizada	Sim
Enviar para lixeira	Nao	Area autorizada	Sim
Restaurar	Nao	Nao	Sim
Excluir definitivamente	Nao	Nao	Sim
Gerenciar usuarios/roles	Nao	Nao	Sim
Gerenciar permissoes	Nao	Nao	Sim
Analytics/relatorios	Nao	Nao	Sim
Auditoria geral	Nao	Nao	Sim

7.1 Cadastro

- Cadastro publico cria somente aluno.
- Email e unico.
- Professor/admin sao atribuidos somente por admin.
- Usuario desativado nao autentica.

7.2 Sessao

Preferencia V1: sessao com token opaco forte, hash do token no MySQL e cookie HttpOnly/Secure/SameSite. Segredos de sessao nao devem ser armazenados em localStorage. Requisicoes de alteracao de estado devem ter protecao CSRF.

7.3 Recuperacao de senha

- Disponivel para aluno, professor e admin.
- Token aleatorio, unico, temporario e de uso unico.
- Banco armazena somente hash do token.
- Resposta publica nao revela se o email existe.
- Nova senha invalida o token e pode revogar sessoes anteriores.
- Senha/token nunca aparecem em logs.

8. Professor

Professor possui liberdade operacional somente dentro das areas concedidas pela administracao.

- upload de PDF/video;
- escolha de destino autorizado;
- edicao de titulo, descricao, tags e outros metadados;
- movimentacao entre areas autorizadas;
- substituicao por nova versao;
- envio para lixeira.

Professor nao pode acessar analytics, relatorios, auditoria geral, gestao de usuarios ou gestao de permissoes.

9. Admin

- gestao de usuarios e roles;
- gestao de categorias;
- gestao de permissoes de professor;
- gestao da lixeira;
- restauracao e exclusao definitiva;

- analytics e relatorios;
- auditoria geral;
- todas as operacoes de material.

10. Materiais e metadados

Tipos suportados na V1: PDF e video.

- Arquivo fisico no Google Drive.
- Registro correspondente no MySQL.
- Referencia externa por `drive\_file\_id`.
- Classificacao por categoria/subcategoria.
- Filtros por disciplina, concurso, professor, tipo e tags quando aplicavel.

10.1 Upload seguro

- autenticacao obrigatoria;
- autorizacao de area no backend;
- validacao real de MIME/extensao;
- limite de tamanho configuravel;
- nome seguro;
- arquivo temporario removido em sucesso/falha;
- consistencia entre Drive e MySQL;
- tratamento de falhas parciais;
- teste com videos representativos no ambiente Hostinger antes do go-live.

10.2 Lixeira

Professor pode enviar material para lixeira quando possuir permissao na area. Restauracao e exclusao definitiva ficam reservadas ao admin. Toda exclusao definitiva deve ser auditada.

11. Analytics administrativo

Analytics e exclusivo do admin.

- alunos ativos por dia;
- acessos por dia;
- evolucao semanal e mensal;
- visualizacoes/reproducoes;
- downloads;
- buscas e termos mais pesquisados;
- materiais mais acessados;
- categorias/disciplinas/concursos mais acessados;
- horarios de maior uso.

Como o aluno tera login, a preferencia e contar usuarios distintos pelo identificador interno. Nao armazenar IP bruto indefinidamente apenas para analytics nem coletar dados pessoais sem necessidade.

12. Banco de dados

- MySQL + InnoDB + utf8mb4.
- Foreign keys, constraints e indices coerentes.
- SQL parametrizado.
- Migrations versionadas.
- Transacoes nas operacoes criticas.
- Nada de alteracoes manuais em producao como processo oficial.

Entidades previstas para modelagem:

- usuarios e sessoes;
- recuperacoes de senha;
- categorias;

- disciplinas e concursos;
- materiais e associacoes;
- tags;
- permissoes de professor;
- analytics;
- auditoria;
- sincronizacoes do Drive.

13. Ciberseguranca

Seguranca e requisito transversal. O sistema nao pode depender de controles visuais do frontend para proteger operacoes.

- autenticao e autorizacao no backend;
- principio do menor privilegio;
- rate limit em login/cadastro/recuperacao;
- CSRF;
- protecao XSS;
- SQL parametrizado;
- headers HTTP de seguranca;
- cookies seguros;
- validacao de upload;
- secrets somente em ambiente seguro;
- logs sem segredo;
- auditoria;
- backup;
- revisao de dependencias.

13.1 Testes obrigatorios antes de producao

- bypass de login;
- escalacao aluno -> professor/admin;
- escalacao professor -> admin;
- IDOR;
- SQL Injection;
- XSS;
- CSRF;
- brute force;
- enumeracao de email;
- mass assignment;
- MIME spoofing e upload malicioso;
- arquivo grande;
- acesso a pasta nao autorizada;
- chamada direta de endpoint admin;
- manipulacao de IDs do Drive;
- exposicao de secrets;
- dependencias vulneraveis conhecidas;
- cookies, CORS e cache privado.

14. Padrao de codigo

- CommonJS: require/module.exports.
- Nao usar arrow functions no codigo de negocio.
- Funcoes tradicionais nomeadas.
- Variaveis/funcoes de negocio em portugues.
- Sem abreviacoas obscuras.
- Controllers leves.
- Regra de negocio em services.
- SQL em repositories/models.
- Google Drive em provider isolado.
- Codigo simples e legivel antes de codigo compacto.

Exemplo de estilo preferido:

```
function verificarPermissaoDeAdmin(req, res, next) {  
  const usuario = req.usuario;  
  const papelDoUsuario = usuario.papel;  
  
  if (papelDoUsuario !== "admin") {  
    throw new AppError("Usuario sem permissao", 403);  
  }  
  
  next();  
}
```

15. UX

A referencia recebida deve ser usada apenas como conceito de organizacao, nunca como copia de design.

- portal inicial do acervo;
- busca global em destaque;
- acessos rapidos;
- disciplinas, concursos e professores;
- file manager moderno nas telas internas;
- breadcrumb;
- cards/pastas;
- filtros;
- responsividade.

Paleta orientadora: grafite, branco e turquesa (#45C9C1), usando o turquesa como acento.

16. Metodologia de implementacao

Fase	Objetivo
Fase 0	Documentacao e preparacao.
Fase 1	Estrutura do projeto, configuracoes, banco-base e fundacoes de seguranca.
Fase 2	Cadastro, login, sessoes, roles e recuperacao de senha.
Fase 3	Categorias, disciplinas, concursos, metadados e permissoes de professor.
Fase 4	Google Drive e importacao/indexacao inicial.
Fase 5	Consulta do acervo, busca, filtros, visualizacao, video e download.
Fase 6	Upload, edicao, movimentacao, substituicao e lixeira.
Fase 7	Painel admin, usuarios, permissoes, analytics e auditoria.
Fase 8	Acabamento frontend, responsividade e UX.
Fase 9	QA, ciberseguranca, carga, deploy e smoke de producao.

Fluxo obrigatorio: ANALISAR -> PLANEJAR -> IMPLEMENTAR -> TESTAR -> RELATAR -> PARAR.

17. Relatorio obrigatorio do Codex

- resumo;
- arquivos criados/alterados;
- migrations e banco;
- endpoints;
- regras de negocio;
- permissoes;
- integracoes;
- testes automatizados e manuais;
- testes de autorizacao e seguranca;
- erros encontrados/corrigidos/abertos;
- riscos;
- impactos;
- divergencias da documentacao;
- sugestoes de backlog;
- estado: PRONTA PARA VALIDACAO ou NAO PRONTA.

O Codex deve parar apos o relatorio. A proxima fase somente com autorizacao do responsavel.

18. Pendencias tecnicas antes/durante as primeiras fases

- criar/configurar projeto Google Cloud e Drive API;
- validar menor escopo OAuth suficiente;
- definir URL/subdominio final;
- definir remetente SMTP para recuperacao de senha;
- testar limite pratico de upload de video na Hostinger;
- criar repositório e inserir a pasta docs.

19. Regra de continuidade

Se houver troca de conta ou chat do Codex, o novo agente nao deve confiar na memoria anterior. Ele deve reler `/docs`, consultar `ESTADO\_ATUAL\_DO\_PROJETO.md`, analisar o repositório e somente entao continuar.

A fonte de verdade e: documentacao versionada + codigo + migrations + relatorios de fase.